

Dispense di Gestione Dei Dati

Marco Galletti

A.A. 2024/2025

Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Fondamenti di Intelligenza Artificiale e Gestione Dei Dati con la professoressa Antonella Poggi del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

Indice

1	Introduzione	1
2	Modello relazionale	2
2.1	Struttura dei dati relazionali	2
2.2	Algebra relazionale	3
2.2.1	Operatori di ridenominazione, selezione e proiezione	4
2.2.2	Il problema del mondo chiuso	5
2.2.3	Operatori di join	6
2.2.4	Informazione incompleta	7
2.2.5	JOIN naturale esterno	7
2.3	Vincoli di integrità	8
2.3.1	Superchiave	9
2.3.2	Chiave primaria	9
3	Il Linguaggio SQL	10
3.1	Introduzione	10
3.2	Interrogazioni di Base (DQL)	10
3.3	Funzioni di Aggregazione e Raggruppamento	16
3.4	Operazioni sugli Insiemi di Tuple	19
3.5	Definizioni dei Dati (DDL)	20
3.6	Manipolazione dei Dati (DML)	22
3.7	Interrogazioni Complesse (Annidate)	23
3.8	Altri Aspetti di SQL	25
4	La progettazione concettuale	26
4.1	Modello Entità-Relazione	26
4.2	La nozione di relazione ISA tra entità	27
4.3	Vincoli di integrità nel modello ER	28

1 Introduzione

I dati sono onnipresenti nella vita di ogni persona. Dalla gestione dei conti correnti, all'acquisto dei prodotti online ai social network. Sono vere e proprie risorse per un'organizzazione. Le funzioni di un **sistema informativo** sono la raccolta, l'archiviazione, la manipolazione e la distribuzione dei dati. Il **sistema informatico** è un sistema che gestisce i dati in maniera automatizzata. Quando

si parla di base di dati si intende una collezione dati gestita da un DBMS (Database Management System). Abbiamo un architettura a tre livelli del software:

- **Livello della presentazione:** si occupa di come i dati vengono presentati all'utente (al client k -esimo viene inoltrata l'applicazione k -esima)
- **Livello logico:** application server, si occupa di come i dati vengono manipolati dall'applicazione
- **Livello dei dati:** si occupa di come i dati vengono memorizzati nel DBMS

Il sistema deve essere in grado di gestire collezioni di dati che siano:

- **Grandi:** dati che superano la capacità di memoria di un singolo computer
- **Persistenti:** dati che devono essere mantenuti anche dopo che il programma è terminato
- **Condivisi:** dati che possono essere accessibili da più utenti contemporaneamente

e deve garantire:

- **Affidabilità:** resistenza a malfunzionamenti hardware e software
- **Privatezza:** protezione dei dati dagli accessi non autorizzati
- **Efficienza:** velocità di accesso ai dati
- **Efficacia:** rendere produttive le attività dei suoi utilizzatori

2 Modello relazionale

2.1 Struttura dei dati relazionali

Si basa sul concetto di **relazione**, che è un insieme di tuple (righe) e di attributi (colonne). La relazione matematica sta nel fatto che dati n insiemi $D_1, D_2, \dots, D_n$, il loro prodotto cartesiano è l'insieme di tutte le n -uple ordinate $(d_1, d_2, \dots, d_n)$ dove $d_i \in D_i$ per ogni $i = 1, 2, \dots, n$.

Una relazione matematica su n domini $D_1, D_2, \dots, D_n$ è un sottoinsieme del prodotto cartesiano $D_1 \times D_2 \times \dots \times D_n$.

Il numero di n -uple in una relazione è detto **cardinalità** della relazione.

Se R è una relazione matematica sugli insiemi $D_1, D_2, \dots, D_n$, tali insiemi sono detti **dominio** di R . Una relazione su n domini si dice che ha grado n (o arità n).

Definizione 2.1 (Relazione matematica). Una relazione matematica è un insieme di n -uple ordinate su domini $D_1, D_2, \dots, D_n$, dove ogni tupla è una sequenza di valori della forma $(d_1, d_2, \dots, d_n)$ con $d_i \in D_i$ per ogni $i = 1, 2, \dots, n$. Essendo la relazione un insieme, non ha importanza l'ordine delle n -uple e non possono esserci tuple duplicate.

Ciascuna tupla di una relazione è **ordinata** nel senso che il valore di d_i è associato al dominio D_i .

La relazione nel modello relazionale estende il concetto matematico di relazione. Infatti:

- Ogni relazione ha un nome e un insieme di attributi, uno per ogni dominio del prodotto cartesiano (nome delle colonne)
- Ogni attributo ha un nome e un dominio. Il valore atomico è un valore semplice (intero, stringa, booleano, etc.)
- Non può succedere che due attributi di una stessa relazione abbiano lo stesso nome

Data la relazione $R \in X$, se t è una tupla di R , allora $t[A]$ (o $t.A$) è il valore di t associato all'attributo A .

La stessa notazione la estendiamo a liste di attributi, per esempio $t[A_1, A_2, \dots, A_n]$ è la lista di

valori di t associati agli attributi $A_1, A_2, \dots, A_n$.

Se t è una tupla di R con attributi $A_1, A_2, \dots, A_n$ e si ha che:

$$t[A_1] = a \quad t[A_2] = b \quad \dots \quad t[A_n] = c$$

allora t si può denotare come:

$$\langle A_1 : a, A_2 : b, \dots, A_n : c \rangle$$

Definizione 2.2 (Schema di relazione). Data una collezione di dati C si dice schema di C l'insieme delle informazioni che descrivono la struttura dei dati in C .

Lo schema di relazione è costituito dal nome della relazione e dall'insieme degli attributi, ognuno caratterizzato da un nome e un dominio.

Se R è il nome della relazione e $X = \{A_1, A_2, \dots, A_n\}$ è l'insieme degli attributi e $D_1, D_2, \dots, D_n$ sono i rispettivi domini, lo schema di relazione si denota come $R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$.

Lo **schema di una base di dati** è costituito dal nome della base di dati e dall'insieme degli schemi delle relazioni che la compongono. Se B è il nome della base di dati e $S = \{S_1, S_2, \dots, S_n\}$ è l'insieme degli schemi delle relazioni che la compongono, lo schema di B si denota come $B = \{S_1, S_2, \dots, S_n\}$.

Definizione 2.3 (Istanza di relazione). Un'istanza di relazione su uno schema di relazione $R(X)$ è un insieme di tuple di attributi X .

Definizione 2.4 (Istanza di base di dati). Un'istanza di base di dati su uno schema $B = \{S_1, S_2, \dots, S_n\}$ è un insieme di istanze di relazione $r = \{r_1, r_2, \dots, r_n\}$ dove r_i è un'istanza di relazione su S_i per ogni $i = 1, 2, \dots, n$.

Per uno schema di relazione (o di base di dati) esistono più istanze di relazione che soddisfano lo schema.

2.2 Algebra relazionale

Operazioni di base:

- Operazioni sullo schema: **DDL** (Data Definition Language)
- Operazioni di manipolazione dei dati: **DML** (Data Manipulation Language - interrogazioni)

I linguaggi di interrogazione per basi di dati relazionali si dividono per tipologie:

- Dichiarativi: descrivono le proprietà delle relazioni che si vuole ottenere
- Procedurali: descrivono come ottenere le relazioni che si vuole ottenere

Gli operatori dell'algebra relazionale:

- sono definiti su relazioni
- producono relazioni come risultato
- possono essere composti per ottenere espressioni più complesse

Sono **insiemistici** (unione, intersezione, differenza), ridenominazione, selezione, proiezione, join in diverse forme (join naturale, prodotto cartesiano, theta join).

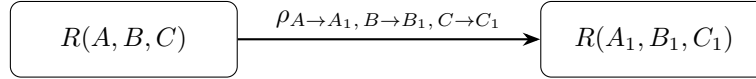
- a livello **estensionale**, le relazioni sono insiemi di tuple, quindi è sensato definire per esse gli operatori insiemistici
- sono operatori **binari** e i risultati sono a loro volta **relazioni** (proprietà di **chiusura**)
- è possibile applicare unione ($\cup$), intersezione ($\cap$), differenza ($\setminus$) solo a relazioni con **stessi attributi**
- la semantica degli operatori insiemistici è quella classica

Osservazione 2.1. *L'algebra relazionale è un impianto matematico formato da operatori su relazioni. Il risultato di ogni operazione è esso stesso una relazione. Di questa relazione sono rilevanti l'insieme degli attributi e le tuple.*

Al contrario quando analizzeremo i sistemi di gestione delle basi di dati, ci occuperemo di come questi sistemi gestiscono le relazioni.

2.2.1 Operatori di ridenominazione, selezione e proiezione

Definizione 2.5 (Operatore: ridenominazione). L'operatore di ridenominazione è un operatore monadico (un solo argomento) che permette di cambiare il nome degli attributi di una relazione, per cui modifica lo schema.



Osservazione 2.2. Non è possibile utilizzare l'operatore di ridenominazione per nominare un attributo con lo stesso nome di un altro.

La sintassi dell'operatore di ridenominazione è:

$$\text{REN}_{\{A \rightarrow A_1, B \rightarrow B_1, \dots, C \rightarrow C_1\}}(R) = \rho_{A \rightarrow A_1, B \rightarrow B_1, \dots, C \rightarrow C_1}(R)$$

Paternità

Padre	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

$\text{REN}_{\text{Genitore} \leftarrow \text{Padre}}$ (Paternità)

Genitore	Figlio
Adamo	Abele
Adamo	Caino
Abramo	Isacco

Figura 1: Operatore di ridenominazione

Definizione 2.6 (Operatore: selezione). L'operatore di selezione è un operatore monadico che prevede la formula di selezione come argomento (espressione booleana). Produce un risultato che:

- ha lo stesso schema della relazione di partenza
- contiene un sottoinsieme delle tuple della relazione di partenza, quelle che soddisfano la formula di selezione

La sintassi dell'operatore di selezione è:

$$\text{SEL}(\phi, R) = \text{SEL}_{\phi}(R)$$

dove ϕ è l'espressione booleana e R è la relazione di partenza.

La **selezione**, assieme alla **proiezione**, è un operatore ortogonale, cioè non modifica lo schema delle relazioni su cui opera.

Impiegato

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

⇒

Impiegato

Matricola	Cognome	Filiale	Stipendio
5998	Neri	Milano	64

$\text{SEL}_{\text{Stipendio} > 50}$ (Impiegato)

Figura 2: Operatore di selezione

Definizione 2.7 (Operatore: proiezione). L'operatore di proiezione è un operatore monadico che prevede l'elenco degli attributi come argomento. Produce un risultato che:

- ha lo stesso schema della relazione di partenza
- contiene un sottoinsieme delle tuple della relazione di partenza, ottenute eliminando gli attributi non elencati

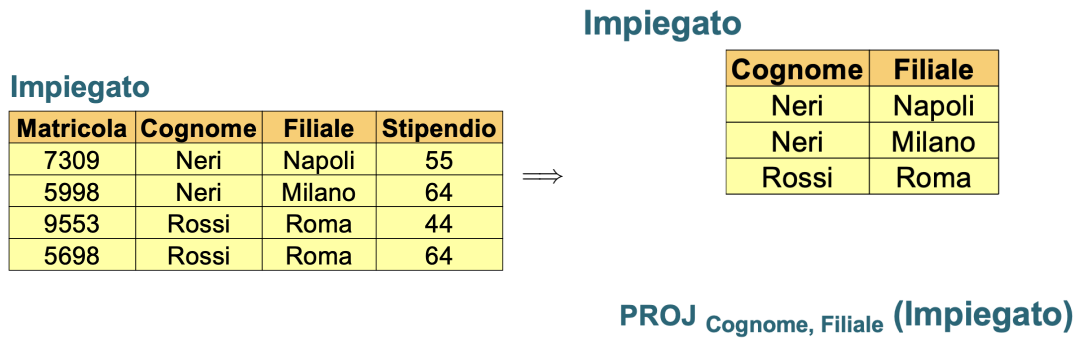


Figura 3: Operatore di proiezione

La sintassi dell'operatore di proiezione è:

$$\text{PROJ}(A_1, A_2, \dots, A_n, R) = \text{PROJ}_{A_1, A_2, \dots, A_n}(R)$$

dove $A_1, A_2, \dots, A_n$ sono gli attributi e R è la relazione di partenza.

Osservazione 2.3 (Cardinalità delle proiezioni). *Una proiezione:*

- contiene al più tante tuple quante la relazione di partenza (operando)
- può contenere tuple duplicate, pertanto potrebbe essere necessario eliminare le tuple duplicate

Combinando selezione e proiezione possiamo ottenere informazioni che ci interessano da una relazione.

2.2.2 Il problema del mondo chiuso

Nelle basi di dati relazionali vige il principio del **mondo chiuso** (*closed world assumption*), secondo cui:

“Se una tupla non è presente nella base di dati, allora il fatto rappresentato da tale tupla è considerato falso nel mondo corrispondente alla base di dati.”

Questo principio ha importanti implicazioni:

- Una base di dati rappresenta un sistema in cui tutto ciò che non è esplicitamente dichiarato come vero è considerato falso
- Quando formuliamo interrogazioni, assumiamo implicitamente che stiamo operando all'interno di questo mondo chiuso
- Ad esempio, se interroghiamo una base di dati di concerti, assumiamo che non esistano concerti oltre a quelli presenti nella base di dati

Osservazione 2.4. *Esistono sistemi di rappresentazione dell'informazione che operano con un'assunzione di **mondo aperto** (open world assumption), tipicamente utilizzati nell'ambito dell'Intelligenza Artificiale, dove l'assenza di informazione non implica necessariamente falsità ma piuttosto incertezza o mancanza di conoscenza.*

Sottolineiamo una proprietà fondamentale dell'algebra relazionale: l'unico modo per esprimere condizioni di negazione è utilizzare l'operatore di differenza. Da questo segue un principio importante:

“L'insieme degli elementi di A che non soddisfano la proprietà B è uguale all'insieme che otteniamo da A togliendo gli elementi che soddisfano la proprietà B”

2.2.3 Operatori di join

Definizione 2.8 (Operatore: Join). L'operatore di join è un operatore binario che prevede due relazioni e una formula di join come argomenti. Esistono diversi tipi di join: join naturale, join theta, equijoin, join esterno...

- **Join naturale:** produce un risultato il cui schema ha l'unione degli attributi degli operandi (relazioni). Le tuple sono costruite dalle tuple degli operandi che si "combinano", una da ciascun operando.

$$t \in X_1 \cup X_2 \mid \exists \text{ due tuple } t_1 \in X_1 \text{ e } t_2 \in X_2 \text{ tali che } t[X_1] = t_1 \text{ e } t[X_2] = t_2$$

Diciamo che $t_1 \in R_1$ e $t_2 \in R_2$ sono **combinabili** dal join naturale se $t_1[X_1 \cap X_2] = t_2[X_1 \cap X_2]$. Un join naturale si dice **completo** se ogni tupla di R_1 è combinabile con ogni tupla di R_2 . Può anche accadere che il risultato di un join naturale sia vuoto. In generali il join di R_1 e R_2 contiene un numero di tuple compreso tra 0 e $|R_1| \times |R_2|$. Quando ho un join naturale con operandi che non hanno attributi in comune, il risultato è il prodotto cartesiano delle due relazioni.

Prepara		Contiene	
Chef	Ricetta	Ricetta	Ingrediente
1	Tiramisu	Tiramisu	Caffè
3	Crepes	Tiramisu	Uova
1	Tenerina	Tenerina	Latte
2	Tenerina	Tenerina	Uova

Prepara join Contiene		
Chef	Ricetta	Ingrediente
1	Tiramisu	Caffè
1	Tiramisu	Uova
1	Tenerina	Uova
1	Tenerina	Latte
2	Tenerina	Uova
2	Tenerina	Latte

Figura 4: JOIN naturale

Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori

Figura 5: JOIN naturale non completo

Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Rossi	A	Mori
Neri	B	Bruni
Bianchi	B	Bruni

Figura 6: JOIN naturale completo

Osservazione 2.5. Il join naturale è *commutativo* e *associativo*.

- **Theta-join:** può essere utilizzato solo su operandi che **non** hanno attributi in comune.

$$R_1 \text{Join}_{\text{condizione}} R_2 \iff \text{SEL}(\text{condizione} R_1 \text{Join} R_2)$$

Se il solo operatore di confronto nella condizione θ è l'uguaglianza, allora si ha un **equi-join**. Per riformulare un Join naturale usando un equi-join occorre procedere a ridenominazioni. Anche per riformulare un equi-join in un join naturale occorre procedere a ridenominazioni.

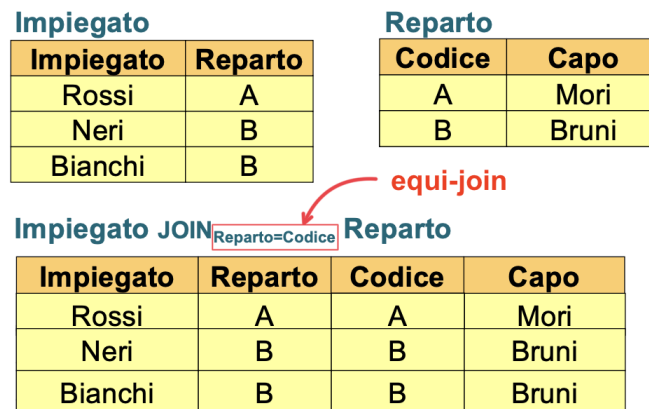


Figura 7: Theta-join

Quando una operazione di join coinvolge la stessa relazione due volte si definisce un **self-join**.

2.2.4 Informazione incompleta

Il modello relazionale impone ai dati una struttura rigida, rappresentata per mezzo di n -uple. Nella pratica non sempre i dati possono corrispondere esattamente allo schema previsto dalla relazione. Nel modello relazionale è introdotto il **valore nullo** NULL che denota l'assenza di un valore dal dominio. Esso può comparire come valore di qualunque attributo definito su qualunque dominio. Distinguiamo tre casi differenti:

- **Valore sconosciuto:** esiste un valore del dominio che non è noto.
- **Valore inesistente:** non esiste un valore del dominio
- **Valore senza informazione:** non è noto se esista o meno un valore del dominio

Non c'è distinzione tra tipi di valore nullo. Una condizione necessaria per la verifica di una proprietà è che l'attributo non sia NULL.

Impiegato

Matricola	Cognome	Filiale	Età
7309	Rossi	Roma	32
5998	Neri	Milano	45
9553	Bruni	Milano	NULL

SEL $Età > 40$ (Impiegato)

Figura 8: Valore nullo

Abbiamo modo di esprimere delle condizioni sul valore nullo:

IS NULL IS NOT NULL

2.2.5 JOIN naturale esterno

JOIN naturale esterno estende, con valori NULL, le tuple che verrebbero tagliate fuori da un join interno. Esiste in tre versioni:

- **LEFT OUTER JOIN**: include tutte le tuple della prima relazione e le tuple della seconda relazione che soddisfano la condizione di join.
- **RIGHT OUTER JOIN**: include tutte le tuple della seconda relazione e le tuple della prima relazione che soddisfano la condizione di join.
- **FULL OUTER JOIN**: include tutte le tuple delle due relazioni e le tuple che soddisfano la condizione di join.

Quindi ad esempio date le seguenti relazioni:

Impiegato		Reparto	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Figura 9: Relazioni

Abbiamo:

Impiegato JOIN_{LEFT} Reparto

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL

Figura 10: LEFT OUTER JOIN

Impiegato JOIN_{RIGHT} Reparto

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
NULL	C	Bruni

Figura 11: RIGHT OUTER JOIN

Impiegato JOIN_{FULL} Reparto

Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

Figura 12: FULL OUTER JOIN

Osservazione 2.6. Due espressioni si dicono **equivalenti** nell'algebra relazionale se producono lo stesso risultato per ogni istanza delle relazioni che partecipano alle espressioni. In pratica l'equivalenza è importante dato che i DBMS cercano sempre di eseguire query equivalenti e meno costose rispetto a quella data.

2.3 Vincoli di integrità

Un vincolo di integrità è una condizione che si esprime a livello di schema e che si intende debba essere soddisfatta in ogni istanza della base di dati, perché individua una condizione necessaria per tutte quelle istanze della base di dati che soddisfano lo schema. Un vincolo esprime una condizione attraverso un'espressione booleana, che deve essere vera nella base di dati. I vincoli di integrità

- Risultano utili a descrivere la realtà in maniera più accurata
- Contribuiscono alla qualità dei dati
- Sono utilizzati dal sistema nella scelta della strategia di esecuzione delle interrogazioni
- Ausilio alla progettazione

Si parla di **istanze ammissibili** per una base di dati per tutte le istanze che soddisfano tutti i vincoli di integrità. I vincoli si possono classificare in:

- **Vincoli intrarelazionali**: si basano solo sulle tuple di una stessa relazione
 - di relazione: esprimono condizioni sui valori di ciascuna tupla di una relazione

- di tupla: esprimono condizioni sui valori di ciascuna tupla di una relazione indipendentemente dalle altre tuple
- di dominio: impone restrizioni sui valori che un attributo può assumere
- **Vincoli interrelazionali:** esprimono condizioni sui valori di tuple di relazioni differenti
 - di integrità referenziale: specificano che i valori di un attributo di una relazione devono essere presenti in un altro attributo di un'altra relazione (detto anche chiave esterna). Di fatto questa relazione si traduce in un'inclusione.

$$R_1[X] \subseteq R_2[Y]$$

la presenza di valori nulli in X è ammessa.

- di inclusione: simile ai vincoli di integrità referenziale, ma l'attributo nella tabella di destinazione non è necessariamente una chiave.

Un attributo obbligatorio è un attributo che non può assumere il valore nullo. È un caso particolare di vincolo di dominio

2.3.1 Superchiave

Se K è un insieme non vuoto di attributi di una relazione si dice che K è una **superchiave** se non esistono due tuple t_1 e t_2 in R tali che $t_1[K] = t_2[K]$. K si dice chiave **chiave** se K è una superchiave minimale in r , ossia se K è una superchiave in r e nessun sottoinsieme proprio di K è una superchiave in r .

Un vincolo di chiave è un'asserzione che specifica che l'insieme K non vuoto di attributi di R gode di queste proprietà:

1. K è una superchiave in ogni istanza ammissibile di R
2. per nessun sottoinsieme proprio non vuoto di K è verificata la proprietà 1

Dato che le relazioni sono insiemi, una relazione non può contenere tuple uguali tra loro. Segue che per ogni relazione R l'insieme dei suoi attributi S è tale per cui non esistono due tuple t_1 e t_2 in R tali che $t_1[S] = t_2[S]$. Poiché l'insieme di tutti gli attributi di R è una superchiave in ogni istanza ammissibile di R , segue che in ogni istanza ammissibile di R esiste almeno una superchiave. E quindi esiste almeno un sottoinsieme di tale insieme che è minimale: per ogni istanza di relazione esiste almeno un insieme di attributi che sono chiave.

Possiamo individuare delle chiavi in 2 modi:

- Individuazione "Top-down"
- Individuazione "Bottom-up"

2.3.2 Chiave primaria

La presenza di valori nulli nelle chiavi può essere problematica:

Matricola	Cognome	Nome	Corso	Nascita
NULL	NULL	Mario	Ing Inf	5/12/78
78763	Rossi	Mario	Ing Civile	3/11/76
65432	Neri	Piero	Ing Mecc	10/7/79
87654	Neri	Mario	Ing Inf	NULL
NULL	Neri	Mario	NULL	5/12/78

Per ogni relazione si sceglie dunque una **chiave primaria**, su cui non sono ammessi valori nulli. Un **vincolo di chiave primaria** è un'asserzione che specifica che l'insieme K non vuoto di

attributi di R forma la chiave primaria di R . Solo un vincolo di chiave primaria è ammesso per ogni relazione.

Osservazione 2.7. *L'esistenza delle chiavi primarie garantisce l'accessibilità a ciascuna tupla della base di dati.*

3 Il Linguaggio SQL

3.1 Introduzione

SQL (originariamente acronimo di Structured Query Language, ma ora considerato un nome proprio) è il linguaggio standard per interagire con basi di dati basate sul modello relazionale[cite: 3]. I sistemi che gestiscono tali basi di dati sono detti **RDBMS** (Relational DataBase Management Systems).

SQL è un linguaggio complesso che include diverse funzionalità, raggruppabili in sottolinguaggi:

- **DDL (Data Definition Language)**: per la definizione dello schema della base di dati (creazione/modifica/eliminazione di tabelle, vincoli, etc.).
- **DML (Data Manipulation Language)**: per inserire, modificare ed eliminare i dati nelle tabelle.
- **DQL (Data Query Language)**: per interrogare i dati (estrarre informazioni).
- **DCL (Data Control Language)**: per gestire i permessi e il controllo degli accessi.

Dal punto di vista logico, un RDBMS organizza i dati in una gerarchia:

- Un'installazione server (es. PostgreSQL) può gestire più **Database**.
- Ogni Database è una collezione di **Schemi**. Uno schema è assimilabile a una base di dati come l'abbiamo definita nel modello relazionale.
- Ogni Schema contiene gli oggetti della base di dati, principalmente le **Tabelle**.
- Per riferirsi a un oggetto (es. una tabella) in uno schema specifico, si usa la notazione `<nomeschema>.<nomeoggetto>`.
- In PostgreSQL, quando si crea un Database, viene creato automaticamente uno schema chiamato `public`. Se non si specifica lo schema, il sistema assume di stia facendo riferimento a `public` (es. `T` viene interpretato come `public.T`). In questo corso, useremo principalmente lo schema `public`.

Osservazione 3.1 (SQL vs Modello Relazionale). *È importante notare una differenza fondamentale: nel modello relazionale, una relazione è un **insieme** di tuple (non ammette duplicati). In SQL, una tabella è definita come un **multiinsieme** di tuple, il che significa che può contenere tuple identiche. Se una tabella SQL possiede una chiave primaria (o un vincolo **UNIQUE**), allora non potranno esserci tuple duplicate e, in tal caso, la tabella si comporta come una relazione matematica. È buona norma definire sempre una chiave primaria per mantenere la coerenza con il modello relazionale. Tuttavia, anche partendo da tabelle senza duplicati, il risultato di alcune interrogazioni SQL potrebbe contenere duplicati.*

3.2 Interrogazioni di Base (DQL)

L'istruzione principale per interrogare una base di dati in SQL è **SELECT**. Prima di vederla in dettaglio, specifichiamo una convenzione sui nomi.

Convenzione sui nomi

Per riferirsi a un attributo specifico di una tabella, si usa la notazione: `NomeTabella.NomeAttributo`. Tuttavia, se il contesto non è ambiguo (ad esempio, se nella query si fa riferimento a una sola tabella, o se tabelle diverse hanno attributi con nomi distinti), si può omettere il nome della tabella e usare solo `NomeAttributo`.

Struttura base dell'istruzione SELECT

La forma elementare dell'istruzione SELECT è:

```
SELECT [DISTINCT] <lista_attributi_o_espressioni>  
FROM <lista_tabelle_con_alias>  
[WHERE <condizione_di_selezione>];
```

Le tre clausole principali sono:

- **SELECT:** Specifica quali colonne (attributi o espressioni calcolate) devono apparire nel risultato. Questa parte è anche detta *target list*. L'opzione **DISTINCT** elimina le righe duplicate dal risultato finale.
- **FROM:** Indica le tabelle dalle quali estrarre i dati. Se vengono elencate più tabelle, concettualmente viene eseguito il loro prodotto cartesiano.
- **WHERE:** Specifica una condizione booleana che le tuple devono soddisfare per essere incluse nel risultato intermedio (prima della proiezione specificata da **SELECT**). È opzionale; se omessa, tutte le tuple del prodotto cartesiano (o della singola tabella) vengono considerate.

Semantica Operativa (Concettuale):

1. **FROM:** Viene calcolato il prodotto cartesiano di tutte le tabelle elencate. Se c'è una sola tabella, si considera quella.
2. **WHERE:** Vengono filtrate le tuple generate al passo 1, mantenendo solo quelle che soddisfano la condizione specificata.
3. **SELECT:** Per ogni tupla superstite, vengono calcolate le espressioni nella target list, producendo le colonne del risultato finale.
4. **DISTINCT** (se presente): Vengono eliminate eventuali tuple duplicate dal risultato ottenuto al passo 3.

Osservazione 3.2 (SQL SELECT vs Algebra Relazionale). *L'istruzione **SELECT** <lista> FROM Tabella WHERE <condizione> è analoga all'espressione dell'algebra relazionale $PROJ_{<lista>}(SEL_{<condizione>}(Tabella))$. Si usa "analoga" e non "equivalente" perché SQL opera su **multinsiemi** (tabelle che possono contenere tuple duplicate), mentre l'algebra relazionale opera su **insiemi** (relazioni senza duplicati). Il risultato di una query SQL può contenere duplicati, a meno che non si usi la parola chiave **DISTINCT**.*

Dettagli sulla clausola SELECT

- **Selezionare tutte le colonne:** Si può usare l'asterisco (*) per selezionare tutti gli attributi delle tabelle coinvolte:

```
SELECT * FROM Persone WHERE eta < 30;
```

Questo equivale a una selezione senza proiezione ($SEL_{condizione}(Tabella)$). Se si omette anche la clausola **WHERE**, **SELECT *** restituisce l'intera tabella (o l'intero prodotto cartesiano se ci sono più tabelle in **FROM**).

- **Eliminare duplicati:** La parola chiave **DISTINCT** si applica a tutte le colonne della target list e rimuove le righe che risultano identiche dopo la proiezione.

```
SELECT DISTINCT cognome, filiale FROM Impiegati;
```

- **Ridenominare colonne:** Si può assegnare un nome (alias) a una colonna nel risultato usando la parola chiave **AS** (che può anche essere omessa):

```
SELECT nome AS nomePersona, reddito AS salario
FROM Persone
WHERE eta < 30;
```

Oppure:

```
SELECT nome nomePersona, reddito salario
FROM Persone
WHERE eta < 30; -- AS omissso
```

Questo è utile per dare nomi significativi alle colonne, specialmente se derivano da espressioni.

- **Espressioni nella target list:** La target list può contenere espressioni SQL arbitrarie che operano sui valori degli attributi delle tuple selezionate. Le espressioni possono includere:
 - Valori costanti (es. 'Gigi', 1, 2.3).
 - Operatori aritmetici (+, -, *, /).
 - Operatori booleani (NOT, AND, OR).
 - Chiamate a funzioni predefinite o definite dall'utente (es. `trunc(valore)`, `now()`, `date_part('year', data)`).

Esempio: Calcolare il reddito semestrale e assegnare un alias.

```
SELECT nome, reddito / 2 AS redditoSemestrale FROM Persone WHERE eta > 50;
```

Esempio con costante e alias:

```
SELECT 'Gigi' AS nomePersona, eta, reddito / 2 AS redditoSemestrale
FROM Persone
WHERE nome = 'Luigi';
```

Dettagli sulla clausola WHERE

La clausola WHERE specifica le condizioni per filtrare le tuple. Una condizione è un'espressione booleana che può essere costruita usando:

- **Operatori di confronto:** =, <> (o !=), <, >, <=, >=.
- **Operatori booleani:** AND, OR, NOT, con uso di parentesi per gestire la precedenza.

```
WHERE reddito > 25 AND (eta < 30 OR eta > 60)
```

- **IN (lista_valori):** Verifica se un valore appartiene a un insieme specificato. NOT IN verifica il contrario.

```
WHERE sede IN ('Milano', 'Brescia')
-- Equivalente a: sede = 'Milano' OR sede = 'Brescia'
```

```
WHERE sede NOT IN ('Milano', 'Brescia')
-- Equivalente a: sede <> 'Milano' AND sede <> 'Brescia'
```

- **LIKE pattern:** Verifica se una stringa corrisponde a un modello specificato. I caratteri jolly principali sono:
 - %: Rappresenta una sequenza qualsiasi di zero o più caratteri.

- `_`: Rappresenta un singolo carattere qualsiasi.

```
WHERE nome LIKE 'A_d%'
-- Nome inizia per 'A', ha 'd' come terza lettera, seguito da qualsiasi cosa
```

- **IS NULL / IS NOT NULL**: Verifica se un valore è nullo o non nullo. Non si può usare `= NULL`.

```
WHERE eta IS NULL
WHERE eta IS NOT NULL
```

Esempio combinato: persone con età maggiore di 40 o età sconosciuta.

```
WHERE eta > 40 OR eta IS NULL
```

Join tra Tabelle

Per combinare informazioni provenienti da più tabelle, si realizza un'operazione di **join**. Il modo più basilare in SQL è:

1. Elencare tutte le tabelle necessarie nella clausola **FROM**, separate da virgole. Concettualmente, questo genera il **prodotto cartesiano** di tutte le tuple delle tabelle elencate.
2. Specificare la **condizione di join** nella clausola **WHERE**. Questa condizione tipicamente confronta i valori degli attributi corrispondenti nelle diverse tabelle (es. `Tabella1.ID = Tabella2.ForeignKeyID`).
3. Aggiungere eventuali altre condizioni di filtro nella clausola **WHERE**, collegandole con **AND**.
4. Specificare le colonne desiderate dal risultato combinato nella clausola **SELECT**.

Esempio: Trovare nome e stipendio degli impiegati che lavorano nel progetto 'XYZ', supponendo di avere tabelle `Impiegati(Matricola, Nome, Stipendio, ProgettoID)` e `Progetti(ID, NomeProgetto, Budget)`.

```
SELECT Impiegati.Nome, Impiegati.Stipendio
FROM Impiegati, Progetti
WHERE Impiegati.ProgettoID = Progetti.ID -- Condizione di JOIN
AND Progetti.NomeProgetto = 'XYZ';      -- Condizione di filtro aggiuntiva
```

La semantica di `SELECT <lista> FROM R1, R2 WHERE <condizione>` è analoga a `PROJ<lista>(SEL<condizione>(R1 × R2))`.

Osservazione 3.3 (Ottimizzazione dei Join). *Sebbene la semantica concettuale preveda il calcolo del prodotto cartesiano seguito da una selezione, i DBMS reali utilizzano algoritmi ottimizzati per eseguire i join in modo molto più efficiente, evitando quasi sempre di materializzare l'intero prodotto cartesiano, che sarebbe estremamente costoso. L'utente si concentra sulla specifica dichiarativa di cosa vuole ottenere, lasciando al DBMS il compito di trovare il modo più efficiente per calcolarlo.*

Alias per le Tabelle

Quando una query coinvolge più tabelle, o la stessa tabella più volte (self-join), diventa utile (e talvolta necessario) assegnare nomi brevi e temporanei alle tabelle, detti **alias**. Si fa nella clausola **FROM** usando **AS** (che può essere omesso):

```
SELECT p.nome, pr.NomeProgetto
FROM Impiegati AS p, Progetti AS pr
WHERE p.ProgettoID = pr.ID AND pr.NomeProgetto = 'XYZ';
```

-- Senza AS:

```
SELECT p.nome, pr.NomeProgetto
FROM Impiegati p, Progetti pr
WHERE p.ProgettoID = pr.ID AND pr.NomeProgetto = 'XYZ';
```

Gli alias sono **obbligatorî** quando si deve fare riferimento alla stessa tabella più volte nella stessa clausola FROM, come nel caso dei **self-join**.

Self-Join: Un join di una tabella con se stessa. È utile per confrontare tuple diverse all'interno della stessa tabella. Ad esempio, trovare coppie di persone con lo stesso reddito.

```
SELECT p1.nome, p2.nome, p1.reddito
FROM Persone p1, Persone p2
WHERE p1.reddito = p2.reddito -- Condizione di join sullo stesso attributo
    AND p1.nome < p2.nome;    -- Condizione per evitare coppie (A, B) e (B, A) e coppie (A, A)
```

In questo caso, p1 e p2 si riferiscono a tuple diverse (o potenzialmente alla stessa tupla, se non ci fosse la condizione `p1.nome < p2.nome`) della tabella `Persone`.

Join Esplicito con JOIN ON

Oltre al metodo "implicito" di specificare i join elencando tabelle nella clausola FROM e le condizioni nella clausola WHERE, SQL offre una sintassi esplicita per i join, considerata più moderna e chiara, specialmente per query complesse.

La sintassi utilizza la parola chiave JOIN nella clausola FROM e ON per specificare la condizione di join:

```
SELECT ...
FROM Tabella1 JOIN Tabella2 ON CondizioneJoin1
    [JOIN Tabella3 ON CondizioneJoin2] ...
[WHERE CondizioniAggiuntive];
```

Questa sintassi separa nettamente le condizioni di join (strutturali, in ON) dalle condizioni di filtro (sui dati, in WHERE).

Esempio 3.1 (Padre e madre). Trovare padre e madre di ogni persona di cui sono noti entrambi. Query con join implicito:

```
SELECT Paternita.figlio, padre, madre
FROM Maternita, Paternita
WHERE Paternita.figlio = Maternita.figlio;
```

Query equivalente con join esplicito:

```
SELECT Paternita.figlio, padre, madre
FROM Maternita JOIN Paternita ON Paternita.figlio = Maternita.figlio;
```

Esempio 3.2 (Persone che guadagnano più del padre). Trovare le persone che guadagnano più dei rispettivi padri, mostrando nome, reddito e reddito del padre. Query con join implicito e self-join:

```
SELECT f.nome, f.reddito, p.reddito AS redditoPadre
FROM Persone p, Paternita t, Persone f
WHERE p.nome = t.padre
    AND t.figlio = f.nome
    AND f.reddito > p.reddito;
```

Query equivalente con join esplicito:

```
SELECT f.nome, f.reddito, p.reddito AS redditoPadre
FROM Persone p JOIN Paternita t ON p.nome = t.padre
      JOIN Persone f ON t.figlio = f.nome
WHERE f.reddito > p.reddito;
```

Nota come la condizione di filtro `f.reddito > p.reddito` rimanga nella clausola `WHERE`, mentre le condizioni strutturali del join sono nella clausola `FROM` con `ON`.

Natural Join

SQL offre anche l'operatore `NATURAL JOIN`. Questo operatore effettua automaticamente un equi-join tra due tabelle basandosi su **tutti** gli attributi che hanno lo **stesso nome** nelle due tabelle.

```
SELECT padre, madre, figlio
FROM Maternita NATURAL JOIN Paternita;
```

Questa query è equivalente alle precedenti (Padre e Madre) perché l'unico attributo in comune è figlio, ed è proprio su quello che vogliamo fare il join.

Osservazione 3.4 (Pericoli del `NATURAL JOIN`). *Il `NATURAL JOIN` può essere comodo ma anche pericoloso. Se le tabelle hanno attributi con lo stesso nome ma significato diverso, o se si aggiunge/rinomina un attributo in una tabella, il join potrebbe produrre risultati inaspettati o smettere di funzionare. In generale, è spesso più sicuro e leggibile usare la sintassi `JOIN ... ON` specificando esplicitamente gli attributi di join.*

Outer Join

I join visti finora (impliciti, `JOIN ON`, `NATURAL JOIN`) sono **inner join** (join interni): restituiscono solo le tuple che hanno una corrispondenza in entrambe le tabelle secondo la condizione di join.

Gli **outer join** (join esterni) estendono questo comportamento, includendo nel risultato anche le tuple di una o entrambe le tabelle che *non* trovano corrispondenza nell'altra tabella. Gli attributi mancanti della tabella non corrispondente vengono riempiti con valori `NULL`.

Esistono tre tipi di outer join:

- **LEFT OUTER JOIN** (o semplicemente **LEFT JOIN**): Mantiene tutte le tuple della tabella a *sinistra* dell'operatore `JOIN`. Per le tuple di sinistra che non hanno corrispondenza a destra, le colonne della tabella di destra vengono riempite con `NULL`.
- **RIGHT OUTER JOIN** (o semplicemente **RIGHT JOIN**): Mantiene tutte le tuple della tabella a *destra* dell'operatore `JOIN`. Per le tuple di destra che non hanno corrispondenza a sinistra, le colonne della tabella di sinistra vengono riempite con `NULL`.
- **FULL OUTER JOIN** (o semplicemente **FULL JOIN**): Mantiene tutte le tuple di *entrambe* le tabelle. Se una tupla di sinistra non ha corrispondenza a destra, le colonne di destra sono `NULL`. Se una tupla di destra non ha corrispondenza a sinistra, le colonne di sinistra sono `NULL`.

La sintassi è simile a quella del join esplicito:

```
SELECT ...
FROM Tabella1 [LEFT | RIGHT | FULL] OUTER JOIN Tabella2 ON CondizioneJoin
...
```

La parola `OUTER` è opzionale.

Esempio 3.3 (Outer Join tra Persone e Paternita). Supponiamo di avere `Persone(Nome, Reddito)` e `Paternita(Padre, Figlio)`. Vogliamo vedere tutte le persone e, se sono padri, i loro figli.

LEFT JOIN: Mantiene tutte le persone. Se una persona non è padre (non compare in `Paternita.Padre`), il figlio risulterà `NULL`.

```
SELECT Persone.Nome, Figlio
FROM Persone LEFT OUTER JOIN Paternita ON Persone.Nome = Paternita.Padre;
```

RIGHT JOIN: Mantiene tutte le tuple di paternità. Se un padre elencato in **Paternita** non esiste nella tabella **Persone** (improbabile con vincoli referenziali, ma possibile), il suo nome risulterà **NULL**. Mantiene anche i padri presenti in **Persone**.

```
SELECT Persone.Nome, Figlio
FROM Persone RIGHT OUTER JOIN Paternita ON Persone.Nome = Paternita.Padre;
```

FULL JOIN: Mantiene tutte le persone e tutte le tuple di paternità. Le persone che non sono padri avranno **Figlio = NULL**. Le tuple di paternità con padri non presenti in **Persone** avranno **Persone.Nome = NULL**.

```
SELECT Persone.Nome, Figlio
FROM Persone FULL OUTER JOIN Paternita ON Persone.Nome = Paternita.Padre;
```

3.3 Funzioni di Aggregazione e Raggruppamento

SQL permette di calcolare valori riassuntivi su insiemi di tuple tramite le **funzioni di aggregazione**.

Funzioni di Aggregazione Principali

Le funzioni di aggregazione più comuni sono:

- **COUNT:** Conta il numero di tuple o valori.
- **SUM:** Calcola la somma di valori numerici.
- **AVG:** Calcola la media di valori numerici.
- **MIN:** Trova il valore minimo.
- **MAX:** Trova il valore massimo.

Queste funzioni operano su un multi-insieme di valori (una colonna o espressione su più righe) e restituiscono un singolo valore.

Gestione dei NULL:

- Tutte le funzioni di aggregazione, *tranne* **COUNT(*)**, ignorano i valori **NULL** nei loro calcoli.
- **COUNT(*)**: Conta il numero totale di tuple selezionate.
- **COUNT(Attributo)**: Conta il numero di tuple in cui **Attributo** ha un valore *non nullo*.
- **COUNT(DISTINCT Attributo)**: Conta il numero di valori *distinti e non nulli* di **Attributo**.
- **SUM(Attributo)** e **AVG(Attributo)**: Calcolano somma e media solo sui valori non nulli. Funzionano solo per tipi numerici.
- **MIN(Attributo)** e **MAX(Attributo)**: Trovano minimo e massimo tra i valori non nulli. Funzionano per tipi su cui è definito un ordinamento (numeri, stringhe, date, etc.).

Utilizzo senza GROUP BY: Se usate nella clausola **SELECT** senza una clausola **GROUP BY**, le funzioni di aggregazione calcolano un singolo valore riassuntivo per tutte le tuple che soddisfano la clausola **WHERE**.

Esempio 3.4 (Statistiche generali sui **Persone**).

```
SELECT COUNT(*) AS NumeroTotalePersone,
COUNT(reddito) AS NumeroRedditiNoti,
AVG(reddito) AS RedditoMedio,
SUM(reddito) AS SommaRedditi,
MIN(eta) AS EtaMinima,
MAX(eta) AS EtaMassima
```

FROM Persone;

Questa query restituirà una singola riga con le statistiche calcolate sull'intera tabella **Persone**.

Raggruppamento con GROUP BY

Spesso si desidera calcolare le funzioni di aggregazione non sull'intera tabella, ma su *gruppi* di tuple che condividono lo stesso valore per uno o più attributi. Questo si ottiene con la clausola **GROUP BY**.

La clausola **GROUP BY** *Attributo1, Attributo2, ...* partiziona le tuple (quelle che hanno superato il filtro **WHERE**, se presente) in gruppi. Ogni gruppo contiene tutte le tuple che hanno la *stessa combinazione* di valori per gli attributi specificati in **GROUP BY**. Successivamente, le funzioni di aggregazione nella clausola **SELECT** vengono applicate *separatamente a ciascun gruppo*.

Regola Fondamentale del GROUP BY: Se una query include una clausola **GROUP BY**, allora ogni elemento nella clausola **SELECT** deve essere:

- Uno degli attributi elencati nella clausola **GROUP BY**.
- Una funzione di aggregazione.
- Una costante o un'espressione che dipende solo dagli attributi di raggruppamento e/o da funzioni di aggregazione.

Questo perché per ogni gruppo viene restituita una sola riga di risultato. Non avrebbe senso chiedere un attributo non di raggruppamento, perché avrebbe valori diversi all'interno dello stesso gruppo.

Esempio 3.5 (Numero di figli per padre). Data la tabella **Paternita(Padre, Figlio)**, calcolare quanti figli ha ciascun padre.

```
SELECT Padre, COUNT(*) AS NumeroFigli
FROM Paternita
GROUP BY Padre;
```

Semantica:

1. Le tuple di **Paternita** vengono raggruppate in base al valore dell'attributo **Padre**. Si forma un gruppo per ogni valore distinto di **Padre**.
2. Per ciascun gruppo, viene applicata la funzione **COUNT(*)**, che conta il numero di tuple nel gruppo (cioè il numero di figli per quel padre).
3. Il risultato contiene una riga per ogni gruppo (ogni padre), con il valore di **Padre** e il risultato di **COUNT(*)**.

Filtrare i Gruppi con HAVING

A volte si desidera filtrare i risultati non a livello di singola tupla (come fa **WHERE**), ma a livello di gruppo, basandosi sul risultato delle funzioni di aggregazione. Questo si fa con la clausola **HAVING**.

La clausola **HAVING** si applica *dopo* che i gruppi sono stati formati e le funzioni di aggregazione sono state calcolate. Specifica una condizione che ogni gruppo deve soddisfare per essere incluso nel risultato finale. La condizione in **HAVING** tipicamente coinvolge funzioni di aggregazione.

Esempio 3.6 (Padri con più di un figlio). Partendo dall'esempio precedente, vogliamo mostrare solo i padri che hanno più di un figlio.

```
SELECT Padre, COUNT(*) AS NumeroFigli
FROM Paternita
GROUP BY Padre
HAVING COUNT(*) > 1; -- Filtra i gruppi dopo l'aggregazione
```

Semantica:

1. Le tuple vengono raggruppate per **Padre** (**GROUP BY**).
2. Per ogni gruppo viene calcolato **COUNT(*)**.

3. Vengono mantenuti solo i gruppi per cui la condizione $\text{COUNT}(\ast) > 1$ è vera (**HAVING**).
4. Viene prodotto il risultato finale con **Padre** e **NumeroFigli** per i gruppi selezionati.

Osservazione 3.5 (**WHERE** vs **HAVING**). È fondamentale distinguere lo scopo delle due clausole:

- **WHERE**: Filtra le *singole tuple* prima che vengano formate le partizioni per il **GROUP BY**. Le condizioni in **WHERE** non possono contenere funzioni di aggregazione.
- **HAVING**: Filtra i *gruppi* dopo che sono stati creati e le funzioni di aggregazione sono state calcolate. Le condizioni in **HAVING** tipicamente contengono funzioni di aggregazione.

Ordine di Valutazione Concettuale (Esteso)

L'ordine concettuale di valutazione di una query SQL con le clausole viste finora è:

1. **FROM** (con eventuali **JOIN**): Identifica le tabelle sorgente e calcola il prodotto cartesiano o il risultato dei join.
2. **WHERE**: Filtra le tuple individuali basate sulle condizioni specificate.
3. **GROUP BY**: Raggruppa le tuple filtrate in base ai valori degli attributi specificati.
4. **HAVING**: Filtra i gruppi basati sulle condizioni specificate (che usano funzioni di aggregazione).
5. **SELECT**: Calcola le espressioni nella target list per ogni gruppo superstite (incluso funzioni di aggregazione e attributi di raggruppamento).
6. **DISTINCT** (se presente): Elimina le righe duplicate dal risultato finale.
7. **ORDER BY** (se presente): Ordina le righe del risultato finale.

Questo è l'ordine logico; il DBMS può ottimizzare l'esecuzione effettiva in modi diversi, purché il risultato finale sia semanticamente equivalente.

Osservazione 3.6 (Omogeneità della Target List con **GROUP BY**). Come specificato nella Regola Fondamentale del **GROUP BY**, la target list (le colonne e le espressioni nella clausola **SELECT**) deve essere "omogenea" rispetto ai gruppi formati. Si possono selezionare solo:

- Gli attributi usati per il raggruppamento (quelli elencati nella clausola **GROUP BY**).
- I risultati di funzioni di aggregazione calcolate su ciascun gruppo (es. $\text{COUNT}(\ast)$, $\text{AVG}(\text{Reddito})$, $\text{MAX}(\text{Eta})$).
- Costanti o espressioni basate esclusivamente sugli attributi di raggruppamento e/o sui risultati delle funzioni di aggregazione.

Questa regola è stringente: non è permesso selezionare un attributo che non faccia parte del raggruppamento, anche se logicamente si sa che quell'attributo possiede un unico valore all'interno di ciascun gruppo. Ad esempio, se si raggruppano i figli per padre (**GROUP BY Padre**), non si può includere direttamente **Persone.Reddito** del padre nella **SELECT**, anche se ogni padre ha un solo reddito registrato nella tabella **Persone**. Per ottenere tale informazione, si dovrebbe ricorrere a funzioni di aggregazione (come $\text{MIN}(\text{p.Reddito})$ o $\text{MAX}(\text{p.Reddito})$, che restituirebbero lo stesso valore se unico) oppure utilizzare tecniche come join con sottoquery o funzioni finestra (window functions), a seconda del DBMS.

Gestione dei Valori NULL nel Raggruppamento

Quando si effettua un raggruppamento su una colonna che può contenere valori NULL (`GROUP BY AttributoConNull`), tutte le righe che presentano NULL in quella colonna vengono raccolte in un **unico gruppo separato**. Le funzioni di aggregazione vengono calcolate anche per questo gruppo "NULL".

Esempio 3.7 (Contare i figli per ogni padre, mostrando un gruppo per padri sconosciuti). Considerando la tabella `Paternita(Padre, Figlio)`, dove `Padre` può essere NULL.

```
SELECT Padre, COUNT(*) AS NumeroFigli
FROM Paternita
GROUP BY Padre;
```

Il risultato di questa query includerà una riga specifica dove il valore della colonna `Padre` è NULL, e `NumeroFigli` indicherà quanti figli hanno un padre non specificato (NULL) nella tabella `Paternita`. Per escludere questo gruppo dai risultati, si può aggiungere una condizione `WHERE Padre IS NOT NULL` prima della clausola `GROUP BY`.

3.4 Operazioni sugli Insiemi di Tuple

SQL fornisce operatori per eseguire operazioni insiemistiche standard (unione, intersezione, differenza) sui risultati di due o più query `SELECT`. Per poter applicare questi operatori, le tabelle risultanti dalle query devono essere **compatibili** (*union-compatible*), il che significa che devono soddisfare due condizioni:

1. Devono avere lo **stesso numero di colonne**.
2. Le colonne corrispondenti, per posizione, devono avere **tipi di dati compatibili** (cioè, deve essere possibile un confronto o una conversione implicita tra i tipi).

I nomi delle colonne nel risultato finale sono tipicamente ereditati dalla prima query dell'operazione.

UNION e UNION ALL

L'operatore `UNION` combina i risultati di due query compatibili restituendo l'unione insiemistica delle righe.

- **UNION**: Restituisce tutte le righe distinte che compaiono nel risultato della prima query, della seconda query, o di entrambe. **Elimina automaticamente le righe duplicate**.
- **UNION ALL**: Restituisce tutte le righe della prima query seguite da tutte le righe della seconda query, **mantenendo eventuali duplicati** sia all'interno di ciascun risultato sia tra i due risultati. È generalmente più veloce di `UNION` perché non effettua il controllo e l'eliminazione dei duplicati.

INTERSECT

L'operatore `INTERSECT` calcola l'intersezione insiemistica dei risultati di due query compatibili.

- **INTERSECT**: Restituisce solo le righe distinte che sono presenti **in entrambi** i risultati delle query. Elimina i duplicati.

Nota: `INTERSECT` potrebbe non essere supportato da tutti i sistemi DBMS (ad esempio, versioni più vecchie di MySQL).

EXCEPT (o MINUS)

L'operatore **EXCEPT** (standard SQL) o **MINUS** (usato da alcuni DBMS come Oracle) calcola la differenza insiemistica tra i risultati di due query compatibili.

- **EXCEPT / MINUS:** Restituisce le righe distinte che sono presenti nel risultato della **prima query** ma **non** sono presenti nel risultato della **seconda query**. L'ordine delle query è significativo. Elimina i duplicati.

Anche **EXCEPT/MINUS** potrebbero non essere supportati da tutti i DBMS.

Precedenza degli Operatori Insiemistici

Quando si combinano più operatori insiemistici (**UNION**, **INTERSECT**, **EXCEPT**) nella stessa espressione senza usare parentesi, l'ordine di valutazione può dipendere dal DBMS. Spesso vengono valutati da sinistra a destra, ma alcuni sistemi potrebbero dare a **INTERSECT** una precedenza maggiore rispetto a **UNION** ed **EXCEPT**. Per evitare ambiguità e garantire l'ordine di esecuzione desiderato, è fortemente consigliato utilizzare le **parentesi** per raggruppare le operazioni.

3.5 Definizioni dei Dati (DDL)

Il Data Definition Language (DDL) è la componente di SQL utilizzata per definire, modificare ed eliminare la struttura (schema) degli oggetti di una base di dati, principalmente tabelle e vincoli. Le istruzioni DDL fondamentali sono **CREATE**, **ALTER** e **DROP**.

Tipi di Dati Fondamentali

La definizione di una tabella richiede la specificazione di un tipo di dato per ciascun attributo. SQL standardizza diversi tipi, sebbene dettagli implementativi possano variare tra DBMS. I tipi comuni includono:

- **Numerici Esatti:** **INTEGER** (o **INT**), **SMALLINT**, **NUMERIC(p, s)**, **DECIMAL(p, s)**.
- **Numerici Approssimati:** **FLOAT(p)**, **REAL**, **DOUBLE PRECISION**.
- **Stringhe di Caratteri:** **CHARACTER(n)** (o **CHAR(n)**, fissa), **CHARACTER VARYING(n)** (o **VARCHAR(n)**, variabile).
- **Date e Orari:** **DATE**, **TIME**, **TIMESTAMP**.
- **Booleani:** **BOOLEAN** (**TRUE**, **FALSE**, **NULL**).

La scelta del tipo appropriato è fondamentale per la corretta memorizzazione e manipolazione dei dati.

Creazione di Tabelle (CREATE TABLE)

L'istruzione **CREATE TABLE** definisce una nuova tabella, specificandone il nome, le colonne con i rispettivi tipi di dati, e opzionalmente i vincoli.

Sintassi Generale:

```
CREATE TABLE NomeTabella (  
    NomeColonna1 TipoDati1 [VincoliDiColonna1 ...],  
    NomeColonna2 TipoDati2 [VincoliDiColonna2 ...],  
    ...  
    [VincoloDiTabella1],  
    [VincoloDiTabella2],  
    ...  
);
```

I vincoli possono essere specificati a livello di colonna (applicabili alla colonna stessa) o a livello di tabella (potenzialmente applicabili a più colonne).

Vincoli di Integrità (CONSTRAINT)

I vincoli impongono regole sui dati per assicurarne coerenza e validità. È buona pratica assegnare nomi espliciti ai vincoli (`CONSTRAINT NomeVincolo ...`) per facilitarne la gestione.

Tipi Principali di Vincoli:

- **NOT NULL:** Impedisce l'inserimento di valori NULL in una colonna. Definito a livello di colonna.

```
NomeColonna TipoDati NOT NULL
```

- **UNIQUE:** Assicura l'unicità dei valori (o combinazioni di valori) in una o più colonne specificate. Permette valori NULL (generalmente considerati distinti tra loro).

```
-- A livello di colonna:
NomeColonna TipoDati UNIQUE
-- A livello di tabella:
CONSTRAINT NomeVincolo UNIQUE (Colonna1 [, Colonna2 ...])
```

- **PRIMARY KEY:** Identifica univocamente ogni riga della tabella. Implica UNIQUE e NOT NULL. Una tabella può avere al massimo una chiave primaria.

```
-- A livello di colonna (per chiave singola):
NomeColonna TipoDati PRIMARY KEY
-- A livello di tabella (per chiave singola o multipla):
CONSTRAINT NomeVincolo PRIMARY KEY (Colonna1 [, Colonna2 ...])
```

- **DEFAULT:** Specifica un valore predefinito per una colonna, utilizzato se non viene fornito un valore esplicito durante l'INSERT.

```
NomeColonna TipoDati DEFAULT ValorePredefinito
```

- **FOREIGN KEY:** Stabilisce una relazione tra tabelle, garantendo l'integrità referenziale. I valori nelle colonne della chiave esterna devono corrispondere a valori esistenti nella chiave primaria (o unica) della tabella referenziata, o essere NULL (se permesso).

```
-- A livello di tabella (forma più comune):
CONSTRAINT NomeVincolo FOREIGN KEY (ColFK1 [, ColFK2 ...])
    REFERENCES TabellaReferenziata (ColRef1 [, ColRef2 ...])
    [ON DELETE AzioneReferenziale]
    [ON UPDATE AzioneReferenziale]
```

Le azioni referenziali (ON DELETE, ON UPDATE) definiscono il comportamento in caso di modifica/eliminazione della riga referenziata. Azioni comuni:

- **NO ACTION / RESTRICT:** Impedisce l'operazione (default implicito spesso).
- **CASCADE:** Propaga l'operazione alle righe referenzianti.
- **SET NULL:** Imposta NULL nelle colonne della chiave esterna referenziante.
- **SET DEFAULT:** Imposta il valore DEFAULT nelle colonne della chiave esterna referenziante.
- **CHECK:** Definisce una condizione booleana che deve essere soddisfatta da ogni riga della tabella.

```
-- A livello di colonna:
NomeColonna TipoDati CHECK (CondizioneSuColonna)
-- A livello di tabella:
CONSTRAINT NomeVincolo CHECK (CondizioneSuRiga)
```

Modifica della Struttura delle Tabelle (ALTER TABLE)

L'istruzione ALTER TABLE modifica la definizione di una tabella esistente.

Operazioni Comuni:

- **Aggiungere una colonna:** ALTER TABLE NomeTabella ADD [COLUMN] NomeColonna TipoDati [Vincoli];
- **Rimuovere una colonna:** ALTER TABLE NomeTabella DROP [COLUMN] NomeColonna [CASCADE | RESTRICT];
- **Modificare una colonna:** La sintassi specifica (ALTER COLUMN, MODIFY COLUMN, etc.) e le operazioni permesse (es. cambio tipo, aggiunta/rimozione DEFAULT, aggiunta/rimozione NOT NULL) variano significativamente tra i diversi DBMS.

```
-- Esempio concettuale (sintassi specifica del DBMS)
ALTER TABLE NomeTabella ALTER COLUMN NomeColonna TYPE NuovoTipo;
ALTER TABLE NomeTabella ALTER COLUMN NomeColonna SET DEFAULT NuovoValore;
ALTER TABLE NomeTabella ALTER COLUMN NomeColonna DROP DEFAULT;
ALTER TABLE NomeTabella ALTER COLUMN NomeColonna SET NOT NULL;
```

- **Aggiungere un vincolo:**

```
ALTER TABLE NomeTabella ADD CONSTRAINT NomeVincolo DefinizioneVincolo;
```

(La DefinizioneVincolo segue la sintassi dei vincoli di tabella vista in CREATE TABLE).

- **Rimuovere un vincolo:** ALTER TABLE NomeTabella DROP CONSTRAINT NomeVincolo;

Eliminazione di Tabelle e Database

- **DROP TABLE:** Rimuove una tabella, i suoi dati, indici e vincoli associati.

```
DROP TABLE [IF EXISTS] NomeTabella [CASCADE | RESTRICT];
```

RESTRICT (default in molti sistemi) impedisce l'operazione se esistono dipendenze (es. viste, chiavi esterne). CASCADE rimuove la tabella e gli oggetti dipendenti. IF EXISTS previene errori se la tabella non esiste.

- **DROP DATABASE:** Elimina un intero database. Operazione irreversibile che richiede privilegi elevati.

```
DROP DATABASE [IF EXISTS] NomeDatabase;
```

Osservazione 3.7 (Catalogo di Sistema / Dizionario dei Dati). *Ogni DBMS mantiene un catalogo di sistema (o dizionario dei dati): un insieme di tabelle speciali contenenti metadati sulla struttura del database stesso (tabelle, colonne, tipi, vincoli, ecc.). Questo catalogo può essere interrogato per ottenere informazioni sullo schema. Lo standard SQL definisce lo schema `information_schema` a questo scopo.*

3.6 Manipolazione dei Dati (DML)

Il Data Manipulation Language (DML) comprende le istruzioni SQL per l'inserimento, l'eliminazione e la modifica dei dati contenuti nelle tabelle di una base di dati.

INSERT: Inserimento di Righe

L'istruzione **INSERT** aggiunge nuove righe a una tabella.

Sintassi Principali:

1. Inserimento di valori espliciti:

```
INSERT INTO NomeTabella [( Attributo1, ..., AttributoN )]  
VALUES ( Valore1, ..., ValoreN )  
      [( Valore1', ..., ValoreN' ), ...];
```

Permette di inserire una o più righe specificando direttamente i valori per le colonne. Se la lista degli attributi è omessa, i valori devono essere forniti per tutte le colonne nell'ordine di definizione della tabella. Se la lista è presente, i valori devono corrispondere in numero e ordine agli attributi specificati; le colonne omesse riceveranno il loro valore **DEFAULT** o **NULL** (se permesso).

2. Inserimento tramite **SELECT**:

```
INSERT INTO NomeTabella [( Attributo1, ..., AttributoN )]  
( SELECT Espressione1, ..., EspressioneN FROM ... WHERE ... );
```

Inserisce nella tabella le righe risultanti dall'esecuzione di un'istruzione **SELECT**. Il numero, l'ordine e i tipi di dati delle colonne prodotte dalla **SELECT** devono essere compatibili con la lista degli attributi di **NomeTabella** (o con tutte le sue colonne se la lista è omessa).

DELETE: Eliminazione di Righe

L'istruzione **DELETE** rimuove righe da una tabella.

Sintassi:

```
DELETE FROM NomeTabella  
[WHERE CondizioneDiSelezione];
```

Rimuove tutte le righe di **NomeTabella** che soddisfano la **CondizioneDiSelezione**. Se la clausola **WHERE** è omessa, tutte le righe della tabella vengono eliminate. Le operazioni di **DELETE** possono attivare le regole definite tramite vincoli di integrità referenziale (es. **ON DELETE CASCADE**).

UPDATE: Modifica di Righe

L'istruzione **UPDATE** modifica i valori degli attributi nelle righe esistenti di una tabella.

Sintassi:

```
UPDATE NomeTabella  
SET Attributo1 = Espressione1,  
    Attributo2 = Espressione2,  
    ...  
[WHERE CondizioneDiSelezione];
```

Per tutte le righe di **NomeTabella** che soddisfano la **CondizioneDiSelezione**, aggiorna i valori degli attributi elencati nella clausola **SET** con i risultati delle rispettive espressioni. Se la clausola **WHERE** è omessa, l'aggiornamento viene applicato a tutte le righe della tabella. Le operazioni di **UPDATE** possono anch'esse attivare regole di integrità referenziale (es. **ON UPDATE CASCADE**).

3.7 Interrogazioni Complesse (Annidate)

SQL permette di costruire interrogazioni complesse annidando istruzioni **SELECT** (subquery) all'interno di altre istruzioni SQL, principalmente nelle clausole **WHERE**, **FROM** e **SELECT**.

Subquery nella Clausola WHERE

Le subquery in WHERE servono a filtrare le righe della query esterna basandosi sul risultato della subquery.

Utilizzo con Operatori:

- **Confronto Scalare** (=, <>, <, >, <=, >=): Richiede che la subquery restituisca un singolo valore (una riga, una colonna).

```
WHERE Attributo = (SELECT SingleValue FROM ... WHERE ...)
```

- **IN / NOT IN**: Verifica l'appartenenza (o non appartenenza) di un valore a un insieme di valori restituito dalla subquery (una colonna, potenzialmente più righe).

```
WHERE Attributo IN (SELECT ColumnSet FROM ... WHERE ...)
```

- **Confronto con Quantificatori** (ANY/SOME, ALL): Si usa con operatori di confronto (=, <>, <, >, etc.) quando la subquery restituisce un insieme di valori (una colonna, potenzialmente più righe).

```
WHERE Attributo > ALL (SELECT ColumnSet FROM ... WHERE ...)
```

```
WHERE Attributo = ANY (SELECT ColumnSet FROM ... WHERE ...) -- Equivale a IN
```

- **EXISTS**: Verifica la non-vuotezza del risultato della subquery. Restituisce TRUE se la subquery produce almeno una riga, FALSE altrimenti. Spesso usata con subquery correlate.

```
WHERE EXISTS (SELECT * FROM ... WHERE CondizioneCorrelata ...)
```

- **Confronto di Tuple**: Permette di confrontare una riga della query esterna con le righe (compatibili) prodotte da una subquery.

```
WHERE (Attr1, Attr2) IN (SELECT ColA, ColB FROM ... WHERE ...)
```

Semantica, Visibilità e Correlazione

- **Valutazione Concettuale**: Per ogni riga candidata della query esterna, la subquery viene (logicamente) valutata e il suo risultato usato per verificare la condizione WHERE.
- **Visibilità (Scope)**: Ogni query definisce un blocco. Un blocco interno può accedere agli alias e alle tabelle definite nel proprio blocco e in quelli esterni che lo contengono. Un blocco esterno non può accedere a definizioni interne. La ridefinizione di un nome in un blocco interno (shadowing) maschera quello esterno.
- **Subquery Correlate**: Una subquery è correlata se contiene riferimenti a colonne o alias della query esterna. La sua valutazione dipende dalla riga corrente della query esterna.

Subquery nella Clausola FROM (Tabelle Derivate)

Una subquery nella clausola FROM definisce una tabella temporanea (tabella derivata o vista inline) il cui contenuto è il risultato della subquery stessa.

Sintassi Base:

```
SELECT ...
FROM (SELECT ColA, ColB FROM ... WHERE ...) AS NomeAliasObbligatorio
[JOIN AltraTabella ON ...]
WHERE ... ;
```

È obbligatorio assegnare un alias alla tabella derivata. Questo costrutto è utile per pre-elaborare o aggregare dati prima di usarli nella query principale.

Clausola WITH (Common Table Expressions - CTE): Fornisce un modo più strutturato e leggibile per definire una o più tabelle derivate nominandole prima della query principale.

```
WITH
  NomeCTE1 AS (SELECT ...),
  NomeCTE2 AS (SELECT ... FROM NomeCTE1 ...) -- CTEs possono riferirsi a precedenti
SELECT ...
FROM NomeCTE1 JOIN NomeCTE2 ON ... ;
```

Le CTE migliorano l'organizzazione di query complesse e permettono anche definizioni ricorsive (non trattate qui in dettaglio).

Subquery nella Lista SELECT

Una subquery può essere usata come espressione nella lista **SELECT** per calcolare un valore di colonna.

Requisito: Deve restituire un singolo valore scalare (una riga, una colonna) per ogni riga elaborata dalla query esterna. Spesso sono subquery correlate.

```
SELECT Attr1, Attr2,
  (SELECT Aggregato FROM ... WHERE CondizioneCorrelata = TabEsterna.ID) AS ColonnaCalcolata
FROM TabellaEsterna
WHERE ... ;
```

3.8 Altri Aspetti di SQL

Clausola CASE

L'espressione **CASE** implementa la logica condizionale all'interno di una query.

Sintassi Generale:

```
CASE
  WHEN Condizione1 THEN Risultato1
  WHEN Condizione2 THEN Risultato2
  ...
  [ELSE RisultatoDefault]
END
```

Valuta le condizioni in ordine e restituisce il risultato corrispondente alla prima condizione vera. Se nessuna condizione è vera, restituisce il risultato specificato in **ELSE** o **NULL** se **ELSE** è omissso. È utilizzabile in varie clausole, come **SELECT** e **ORDER BY**.

CREATE VIEW: Creazione di Viste

Una vista è una tabella virtuale basata sul risultato di una query **SELECT** memorizzata nello schema del database.

Sintassi:

```
CREATE [OR REPLACE] VIEW NomeVista [( Colonna1, Colonna2, ... )] AS
( SELECT QueryCheDefinisceLaVista ... );
```

Le viste non contengono dati propri (salvo casi di viste materializzate) ma eseguono la query sottostante quando vengono interrogate. Servono a semplificare query complesse, fornire un livello di astrazione e implementare meccanismi di sicurezza (limitando righe/colonne visibili). L'aggiornamento dei dati tramite viste è soggetto a restrizioni significative. L'opzione **OR REPLACE** permette di aggiornare la definizione di una vista esistente.

4 La progettazione concettuale

Ha lo scopo di produrre uno schema concettuale che descriva il dominio di interesse sulla base delle necessità dell'applicazione.

Dobbiamo dunque stabilire delle relazioni ‘ragionate’ tra i concetti. La progettazione concettuale si interroga sul **cosa** mi interessa e si passa poi a stabilire il **come** rappresentare questi dati (progettazione logica).

- Schema concettuale: modello Entità-Relazione (ER)
- Schema logico: modello relazionale, DDL (standard SQL)
- Schema fisico: modello relazionale + strutture fisiche

4.1 Modello Entità-Relazione

Un modello di dati è un linguaggio con il quale si esprimono schemi concettuali dei dati. Il modello dei dati è caratterizzato dai costrutti ammessi. Occorre dunque specificare per ogni costrutto le proprietà sintattiche, semantiche e pragmatiche.

Lo **schema** descrive la struttura dei dati (e.g. nel modello relazionale l'intestazione delle tabelle e i domini degli attributi).

Ad ogni schema corrispondono più **istanze**, anche se per ogni istante solo una è quella significativa.

Se S è uno schema, un'istanza di S rappresenta lo stato dei dati in un certo istante.

- **Entità**: è una classe di oggetti che sono di interesse e che hanno esistenza autonoma (e.g. clienti, impiegati, prodotti, ...). Ogni entità ha un nome che la identifica univocamente all'interno dello schema. Un'entità è costituita da un insieme di oggetti, detti **istanze**.

È importante notare che un'istanza di entità non è un valore che rappresenta una proprietà di un oggetto, ma l'oggetto stesso. Un'entità denota dunque un **insieme di oggetti “simili”**, mentre un'istanza denota un oggetto dell'insieme rappresentato dall'entità.

- **Attributo di entità**: è una proprietà locale di un'entità. Associa ad ogni istanza di entità un valore appartenente a un insieme di valori possibili dell'attributo (e.g. interi, caratteri, ...)

Si definisce un'attributo per l'entità E quando si vuole rappresentare una proprietà locale delle istanze di E . Ogni attributo di entità ha un nome che lo identifica univocamente all'interno delle entità.

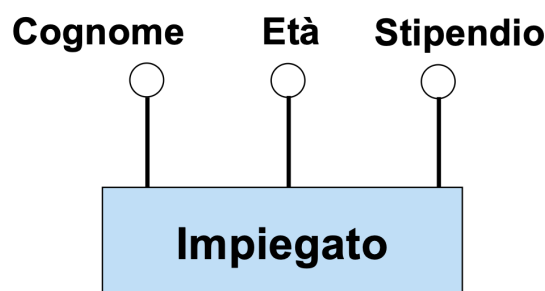


Figura 13: Attributo di entità

La semantica degli attributi è una funzione che associa ad ogni istanza dell'entità un valore:

$$A : \text{Istanze}(I, E) \rightarrow D$$

Un attributo può anche essere definito su un dominio complesso. Un attributo il cui dominio è di tipo record è detto **attributo composto**. Ad esempio, l'attributo ‘indirizzo’ è un attributo composto che può essere scomposto in ‘via’, ‘numero’, ‘cap’.

- **Relazione:** è un legame tra oggetti di entità diverse. Ogni relazione ha un nome che la identifica univocamente all'interno dello schema. Ad esempio se poniamo le entità 'studente' e 'corso' possiamo definire 'EsameSuperato' come relazione tra 'studente' e 'corso'.

Una relazione tra due entità ha come istanze un insieme di coppie etichettate con le entità coinvolte. Non possono esistere due istanze della stessa relazione che coinvolgono le stesse istanze di entità.

Esistono relazioni di arità > 2 e le chiamiamo **relazioni n-arie**.

- **Attributo di relazione:** è una proprietà locale di una relazione, che associa ad ogni istanza di relazione un valore appartenente al dominio dell'attributo. Riprendiamo l'esempio delle entità 'studente' e 'corso' e della relazione 'EsameSuperato', per cui possiamo definire l'attributo 'voto' come attributo di relazione.
- **Ruolo:** aggiunge ad ogni relazione l'informazione che specifica il ruolo di ogni entità coinvolta. La necessità di specificare i ruoli è evidente nel caso di relazioni che coinvolgono la stessa entità. Se si omette di specificare i ruoli, nel caso in cui non siano necessariamente necessari, si assume che il nome del ruolo sia quello dell'entità.

Un concetto viene modellato come:

- **Entità:**
 - Istanze concettualmente significative indipendentemente da altre istanze
 - Se ha o può avere proprietà indipendenti dagli altri concetti
 - Se il concetto è importante
- **Attributo:**
 - Se le sue istanze non sono concettualmente significative
 - Se non ha senso considerare una sua istanza indipendentemente da altre istanze
 - Se serve solo a rappresentare una proprietà locale di un altro concetto
- **Relazione:**
 - Se le sue istanze non sono concettualmente significative indipendentemente da altre istanze (le sue istanze rappresentano n -uple di altre istanze)
 - Se non ha senso pensare alla partecipazione delle sue istanze ad altre relazioni

4.2 La nozione di relazione ISA tra entità

In molti contesti può accadere che tra due classi rappresentate da due entità nello schema concettuale sussista la **relazione ISA**, e cioè che ogni istanza di una entità sia anche istanza di un'altra entità.

La relazione ISA, definita tra due entità, si dicono entità padre ed entità figlia, per cui la seconda rappresenta un tipo di specializzazione della prima (è un suo sottoinsieme).

Osservazione 4.1. *Vige la regola che un'entità può avere al massimo una entità padre. Non è ammessa l'ereditarietà multipla.*

Dalla definizione di relazione ISA segue che essa è **riflessiva** (un'entità è sempre in ISA con se stessa), **transitiva** (se E_1 è in ISA con E_2 e E_2 è in ISA con E_3 , allora E_1 è in ISA con E_3) e **antisimmetrica** (se E_1 è in ISA con E_2 allora E_2 non è in ISA con E_1).

Definizione 4.1 (Principio di ereditarietà). Ogni proprietà di un'entità padre è anche una proprietà della sottoentità, e non si riporta esplicitamente nel diagramma. L'entità figlia può avere proprietà non presenti nell'entità padre.

Una entità può avere diverse entità figlie, e le istanze di due entità che sono figlie di una stessa entità padre possono avere istanze in comune.

Osservazione 4.2. Talvolta l'entità padre è più generale della sottoentità. Talvolta l'entità padre può generalizzare diverse sottoentità rispetto a un unico criterio. Si parla di **generalizzazione**. Nella generalizzazione, le sottoentità hanno insiemi di istanze disgiunti a coppie. Una generalizzazione può essere:

- **completa:** se l'unione delle istanze delle sottoentità è uguale all'insieme delle istanze dell'entità padre
- **Non completa**

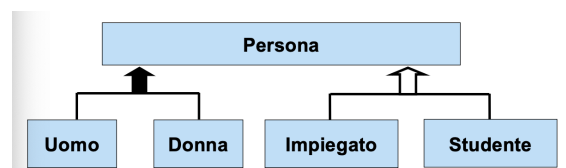
Semantica della generalizzazione Se in uno schema S è definita una generalizzazione tra l'entità padre F e le sottoentità $E_1, \dots, E_n$, in ogni istanza I di S si ha che:

- Ogni istanza di E_i è anche un'istanza di F
- Ogni istanza di F è un'istanza di almeno una delle E_i
- Istanze di E_i e E_j sono disgiunte per ogni $i \neq j$

Se la generalizzazione è completa, si ha che l'unione di tutte le istanze delle sottoentità è uguale all'insieme delle istanze dell'entità padre. Le sottoentità formano una **partizione** dell'entità padre.

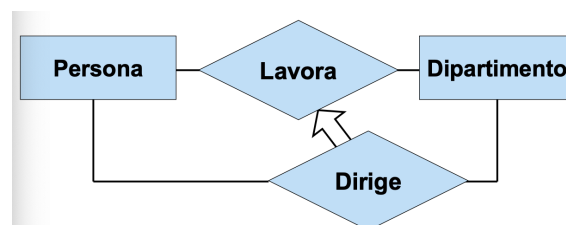
Il principio di ereditarietà si applica anche alle relazioni: ogni proprietà dell'entità padre è anche una proprietà delle sottoentità e non si riporta esplicitamente nel diagramma. Ricordiamo che una entità può avere al più una entità padre.

La stessa entità può essere padre in diverse generalizzazioni.



La relazione ISA e le generalizzazioni possono essere definite anche tra relazioni con la seguente regola: si definisce una relazione ISA tra una relazione R figlia e una relazione Q padre in uno schema S solo se R e Q :

- hanno lo stesso grado (sono tutte e due n -arie)
- hanno gli stessi ruoli (sono coinvolte le stesse entità)
- per ogni ruolo U , l'entità corrispondente a U in R è una entità figlia di quella corrispondente a U in Q



4.3 Vincoli di integrità nel modello ER

Come nel modello relazionale, anche nel modello ER possono essere definiti vincoli di integrità. Ricordiamo che un vincolo di integrità è una regola che si esprime sullo schema e che specifica una condizione che deve essere soddisfatta da tutte le istanze dello schema. Vedremo

- vincoli di cardinalità sulle relazioni

- vincoli di cardinalità sugli attributi
- vincoli di identificazione di entità
- altri vincoli

Vincoli di cardinalità sulle relazioni Un vincolo di cardinalità su associazioni ad un ruolo U (corrispondente ad un'entità E) in una relazione R impone un limite minimo e un limite massimo di istanze della relazione a cui può partecipare ogni istanza di E nel ruolo U .

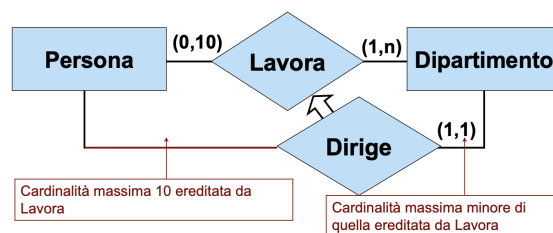
Un vincolo di cardinalità si esprime mediante una coppia x, y associata al ruolo U in R , dove x è il limite minimo e y è il limite massimo. Se assegno $x = 0, y = n$ non ho vincoli di cardinalità né sul minimo né sul massimo.

Tipicamente le cardinalità espresse sono: $0,1,n$. Per la cardinalità minima 0 può indicare partecipazione opzionale, mentre 1 indica partecipazione obbligatoria. Per la cardinalità massima 1 indica che l'entità partecipa al più una volta alla relazione, mentre n indica che l'entità partecipa un numero qualsiasi di volte.

Sussistono inoltre le **classificazioni di relazioni binarie rispetto alla cardinalità**:

- **1:1**
- **1:n**
- **m:n**

La regola sulla ereditarietà della cardinalità massima vale anche se l'entità corrispondente al ruolo U nella relazione figlia è una sottoentità dell'entità corrispondente al ruolo U nella relazione padre.



Se Q ISA R ogni vincolo di cardinalità **massima** definito su R (ruolo U) si applica anche a Q (ruolo U), che potrebbe avere cardinalità massima più stringente di quella ereditata. Questa regola vale anche se l'entità corrispondente al ruolo U nella relazione figlia è una sottoentità dell'entità corrispondente al ruolo U nella relazione padre.

La regola, già introdotta, che asserisce che per un certo ruolo U di una relazione R la cardinalità minima associata ad U deve essere minore o uguale alla cardinalità massima associata ad U rimane valida anche nel caso in cui i vincoli di cardinalità vengono propagati.

Questa regola ci permette di evitare che l'insieme delle istanze di una relazione o di una entità sia sempre vuoto.

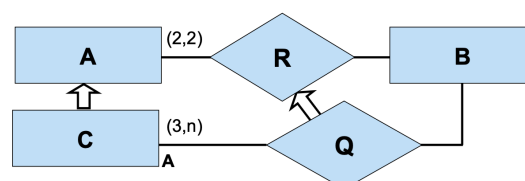


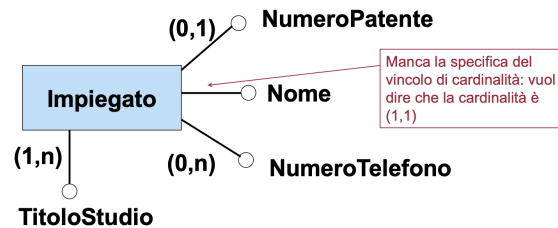
Figura 14: Con questo vincolo, C risulta essere sempre vuoto

Vincoli di cardinalità sugli attributi Definire vincoli di cardinalità sugli attributi ha due scopi:

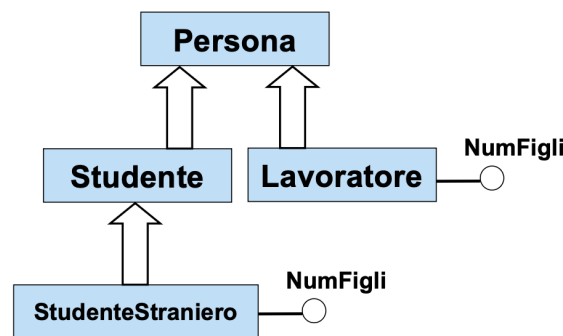
- Indicare **opzionalità** di un attributo
- Indicare **attributi multi-valore**

Quando la cardinalità di un attributo è maggiore di 1, l'attributo si dice **multi-valore** e non rappresenta più una funzione, ma una relazione. Le cardinalità stabiliscono il numero minimo e massimo di valori che l'attributo associa ad ogni istanza dell'entità.

Se la specifica del vincolo manca, si intende che la cardinalità dell'attributo sia (1,1)



Ricordiamo che quando si assegna lo stesso nome a due attributi di entità diverse, si deve ricordare che questi, a livello estensionale, rappresenteranno un'unica relazione.



Vincoli di identificazione di entità Un identificatore di una entità è un insieme di proprietà (attributi o relazioni) che permettono di identificare univocamente le istanze di un'entità. Non esistono due istanze di entità che hanno lo stesso valore per tutte le proprietà che formano l'identificatore.

Un identificatore di una entità E può essere:

- **interno**: formato solo da attributi di E
- **esterno**: formato da attributi di E e da ruoli di relazioni coinvolte in E

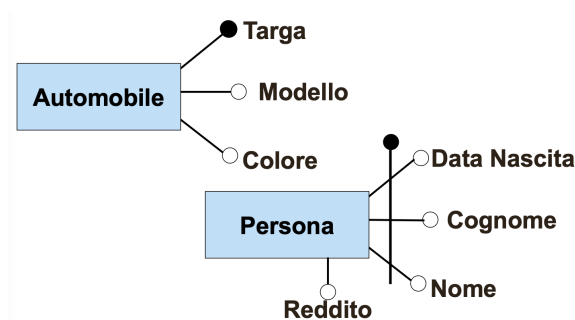


Figura 15: Identificatore interno

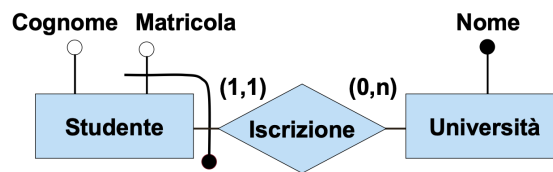


Figura 16: Identificatore esterno

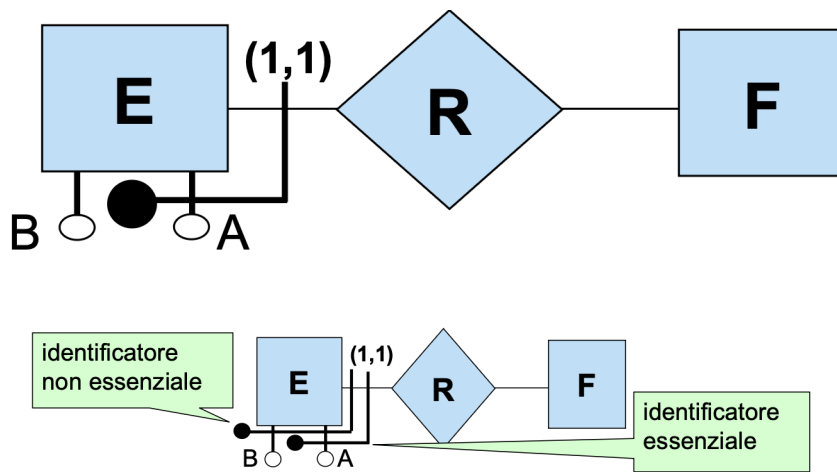
Tutti gli identificatori hanno cardinalità $(1,1)$.

Osservazione 4.3. Sia E una entità che partecipa ad una relazione R nel ruolo U con cardinalità $(1,1)$, e supponiamo che almeno un ruolo U' di R diverso da U abbia cardinalità massima 1. È facile verificare che questa situazione implica che E è identificata esternamente da R tramite U .



Identificatori “essenziali”

Un identificatore dell'entità E formato dall'insieme W di proprietà (attributi o ruoli) si dice **essenziale** se ogni insieme ottenuto da W eliminando un elemento non è più un identificatore di E .



Vincoli di identificazione di relazioni Un identificatore di una relazione R è un insieme di proprietà (attributi o ruoli) che permettono di identificare univocamente le istanze di una relazione. Non esistono due istanze di relazione che hanno lo stesso valore per tutte le proprietà che formano l'identificatore. Per ogni relazione si può dichiarare un numero qualsiasi di identificatori.

Sulla relazione è implicitamente definito un identificatore (implicito) formato dai ruoli della relazione.

Osservazione 4.4 (Minimalità). Si deve evitare il più possibile di rappresentare più volte le stesse proprietà, al fine di garantire una buona progettazione dei dati e facilitarne la manutenzione.

In particolare, distinguiamo due tipologie di ridondanze:

- **Ridondanze intensionali:** si hanno quando la stessa informazione è rappresentata più volte a livello dello schema concettuale, cioè quando si ripetono definizioni o proprietà equivalenti in più punti del modello. Ad esempio, definire separatamente in più entità o relazioni una stessa proprietà come "indirizzo email", invece di riutilizzare un attributo definito una sola volta in un'entità comune.

- **Ridondanze estensionali:** si verificano quando la stessa informazione è rappresentata più volte a livello delle istanze dei dati, ovvero quando un valore specifico appare in diversi record o tuple del database. Ad esempio, se nel database dei dipendenti ogni volta che un dipendente cambia reparto viene duplicata l'intera informazione sul reparto stesso (nome, sede, responsabile), invece di riferirsi a una sola istanza condivisa.

Se una ridondanza estensionale è rilevante dal punto di vista concettuale (ad esempio per ragioni di efficienza nella lettura dei dati), è necessario esplicitarla chiaramente mediante opportuni vincoli di integrità, al fine di preservare la coerenza e la correttezza delle informazioni.